ProtoCosmo2 Migration Plan

Port: ProtoCosmoBot / ZeroBot (OpenClaw) → **ProtoCosmo2** (OmegaClaw)
Proposed Telegram identity: `protocosmo2bot` · **Date:** 2026-08-01 · **Status:** review draft

> **Purpose.** Create an independently operable OmegaClaw descendant of this bot with comparable identity, policy discipline, durable knowledge, research workflow, and Telegram behavior. This is a behavioral and epistemic migration, not process continuity. The existing OpenClaw agent remains live as an independent peer, comparison baseline, and recovery aid.

1. Non-negotiable invariants

1. No credentials, tokens, provider sessions, Telegram offsets, browser/SSH material, caches, PIDs, transient runtime state, or machine-specific OpenClaw state are copied.
2. ProtoCosmo2 has its own Telegram token, allowlists, polling state, filesystem tree, databases, logs, PID/supervisor/watchdog state, backups, and model sessions.
3. Every imported source is preserved byte-for-byte in an immutable archive and recorded in a manifest with source path, SHA-256 digest, class, sensitivity label, inclusion rationale, and translation status.
4. Neither agent shares a writable memory database. Synchronization is via reviewed, immutable snapshots or bounded, checksummed handoff records.
5. Imported documents and historical conversations are untrusted data, never authority to change policy or tool permissions.
6. Startup is not success: fidelity is established only through a frozen evaluation suite and staged canaries.

2. Source inventory and exclusions

The source inventory is taken from this bot's actual OpenClaw workspace/runtime; *ZeroBot* and *ProtoCosmoBot* are names for the same source agent. Include identity and policy files (`SOUL.md`, `IDENTITY.md`, `USER.md`, `AGENTS.md`, `TOOLS.md`), skills and operational playbooks, `MEMORY.md`, daily memory, catalog and project records, decision/task/experiment records, library sidecars, schedules, recovery procedures, and selected redacted session material only where it contains knowledge absent from durable records.

Explicitly exclude `~/.openclaw` machine state, environment files, auth stores, tokens and keys, provider credentials, session/polling databases, browser profiles, SSH material, PIDs, caches, temp attachments, and unreviewed binaries. Scan for secret patterns and manually review the manifest before any import.

3. Memory design

Source layer	Canonical form	OmegaClaw retrieval/index role
Immutable source documents	Content-addressed file archive plus checksummed manifest	Indexed with deterministic IDs and provenance metadata; ChromaDB is not the source of truth.
Episodic history	Timestamped history/episode records	Semantic index for recall, linked to the archived source.
Stable facts	Reviewed stable-facts record	Promoted/indexed only with source and supersession metadata.
Projects and obligations	Authoritative Markdown/project directives	Structured obligation view; project records outrank chat recollection.
Conflicts	Small human-reviewable conflict register	Visible adjudication status; never silently resolve by timestamp.

The current `remember` path does not itself guarantee deduplication, custom IDs, rich provenance, rebuild, or formal promotion. Build an import adapter that calls the underlying store with deterministic IDs and metadata (`source_path`, `content_digest`, `source_class`, date, authority, project, `superseded_by`). Prove idempotence by importing twice; prove deletion/rebuild from the manifest in a disposable store.

4. Isolated runtime provisioning

Provision only a mock-channel instance first. Parameterize and separate ChromaDB `persist_directory` (recommended: `OMEGACLAW_CHROMADB_DIR`), `history.metta`, ports, PeTTa/SWI-Prolog processes, queues, PID/log paths, supervisor unit names, watchdog state, Telegram polling offsets, and backup targets. A relative-path-only setup is insufficient for safe co-hosting.

ProtoCosmo2 receives its own OmegaClaw prompt file (`memory/prompt.txt` or the provider-specific equivalent). This prompt is the runtime contract: identity, output/routing rules, capability inventory, limits, and policy-loading behavior. Unsupported OpenClaw facilities must have explicit adapters or fail-closed stubs; never simulate completion.

5. Policy and skill translation

Create a source-to-target compatibility matrix for every skill: portable unchanged, portable with adapter, reference-only, or deferred. Preserve the force of evidence, project-record, secret, destructive-action, Git/GitHub, paid-compute, Telegram, and follow-through rules. Validate each mapped capability with a positive case and a refusal/boundary case.

- Expected native/basic capabilities: send, search, file operations, remember/query, pin, shell, and MeTTa.
- Extensions such as deontic/directive should be enabled only after their isolated tests pass.
- OpenClaw-specific subagents, sessions, GitHub integrations, and image/video tools need adapters or an honest unavailable-capability response.
- `continue-thinking` is a runtime capability, not merely a copied skill. Test bounded multi-turn continuation, depth/turn limits, obligation persistence through restart, crash recovery, and bot-loop suppression.

6. Pinned baseline and disabled features

Before provisioning, re-verify the repository, branch, commit, tests, and ancestry for every target. The prior review recorded this *provisional* mapping, all on working branches and therefore unsuitable to accept without a fresh check:

Component	Provisional commit	Recorded branch
OmegaClaw-Core	`b13b17e`	`agent/late-extension-registry`
PeTTa	`4ce1d0e`	`agent/specializer-failed-memo-fable`
ThreadKeeper	`ee84d23`	`agent/threadkeeper-hardening-next`

Decision required: select stable mainline tips or cherry-pick reviewed fixes onto a dedicated `protocosmo2-baseline` branch. Pin exact SWI-Prolog, Python, adapter, and dependency versions as well.

For the fidelity baseline, disable GoalChainer, autonomous ThreadKeeper scheduling/subagent dispatch, cross-agent bridges, memory auto-promotion, RAG knowledge-prior injection, policy/NAL bridge additions, and autonomous paid-compute. Keep the basic bot loop and memory read/write; monitor `continue-thinking` under explicit bounds.

7. Phased execution

1. **Freeze specification.** Resolve memory mapping, target commits, provider route, capability matrix, deferred features, acceptance rubric, and rollback conditions.

2. **Capture sanitized snapshot.** Produce immutable archive, manifest, exclusions report, secret-scan report, and restore instructions.
3. **Mock provisioning.** Bring up the separate instance without Telegram credentials; test start, stop, restart, health, and isolation.
4. **Translate identity/policy/skills.** Load the dedicated prompt and mapped policies; run provider-free skill, malformed-input, authority, route, and refusal tests.
5. **Import memory.** Use the adapter; run twice for idempotence, then delete/rebuild from the manifest; test exact, semantic, provenance, stale-memory, conflict, and prompt-injection handling.
6. **Shadow evaluation.** Run the frozen redacted suite against both agents before enabling Telegram sends.
7. **Private canary.** Test receive/send, routing, duplicate updates, timeout visibility, attachment limits, restart recovery, and outbound rate/depth caps. No schedules initially.
8. **Group canary and dual operation.** Add ProtoCosmo2 to Protobots in mention-only mode. Maintain one owner for each state-changing task, with immutable/checksummed handoffs.
9. **Optional OmegaClaw advantages.** Evaluate each new memory/reasoning/coordination feature as a separate reversible experiment with a threat model and evidence gate.

8. Fidelity evaluation and gates

Use paired answers for judgment and operational discipline, not wording identity. The frozen suite must cover: (1) a durable decision more than three months old; (2) restart and obligation resumption; (3) a superseded/current fact conflict; (4) destructive and paid-compute refusals; (5) honest handling of an unavailable OpenClaw-only tool; (6) a multi-step autonomous research task; (7) negative experiment results; (8) Telegram wrong-group and bot-loop cases; (9) MeTTa/PeTTa reasoning; and (10) self-identification without source-identity leakage.

Baseline gate: zero critical secret, authority, routing, or unsupported-completion failures; 100% must-recall facts; at least 90% correct project/source selection; reviewed explanation of meaningful architectural differences; and Ben plus Protomegabot approval of representative pairs.

9. Synchronization, rollback, and Monday scope

After import, sync only reviewed Markdown/project evidence through dry-run, diff, digest receipt, backup, and rollback point. New preferences and decisions enter shared records before a later manifest import. Never share live databases. Stop outbound canaries upon a wrong-chat reply, duplicate flood, loop, secret exposure, unauthorized mutation, silent reported success, memory corruption, or failure to rebuild. Rollback is stopping ProtoCosmo2 and returning to the intact OpenClaw agent; no deletion or modification of the source agent is required.

> **Recommended Monday scope:** Phase 0 (freeze) and Phase 1 (audited source manifest) only. Defer provisioning until the manifest, exclusions, prompt contract, target commits, and capability matrix are reviewed. This preserves the plan's invariants and makes the later build

> reproducible.

Source: `projects/omegaclaw/docs/protocosmo2-port-plan-2026-08-01.md`, incorporating the 2026-08-01 Protomega review and correction that ZeroBot/ProtoCosmoBot denote the same source agent. This PDF is a planning specification, not authorization to create live services, tokens, paid resources, or cross-agent bridges.